Task 03 - The Agent Working Loop

Handbook reference: Page 3 of 10 | Report date: 2026-10-13 | Folder: `task-03-agent-working-loop/`

1. Concept from the handbook

The agent works in a loop: Task -> Inspect -> Propose -> Policy check -> Execute -> Observe -> Adjust -> Verify -> Handoff. The model proposes, the orchestration layer checks policy, tools execute, a human sets intent and approval limits. Retries must be bounded - repeating the same failing action only adds cost and confusion.

2. What was built

A working loop on a real buggy Task Tracker copy. A deterministic **ScriptedModel** (stand-in for an LLM, clearly labelled) reads the file, proposes a dangerous `rm -rf` that the **Policy** blocks, makes a first edit, **observes a real failing test**, adjusts, re-runs and verifies. A 'stuck' model that repeats itself is escalated.

3. Behaviours demonstrated

- Dangerous / out-of-scope proposals are blocked, not executed
- First attempt fails for real -> observation drives the adjustment
- VERIFY only passes if the latest test run is green
- Repeated identical action -> ESCALATE (bounded retries)
- Original workspace is untouched (works in a temp copy)

4. Files in this task

`agent_loop.py`, `demo.py`, `test_agent_loop.py`, `workspace/test_tracker.py`, `workspace/tracker.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

5. Code screenshots

agent_loop.py

```python
"""Task 03 - The agent working loop.

Task -> Inspect -> Propose -> Policy check -> Execute -> Observe -> Adjust -> Verify -> Handoff

IMPORTANT: ScriptedModel is a deterministic stand-in for an LLM so the demo is
reproducible offline. The loop, policy, tools and observations are real: files
really change and tests really run. Swap ScriptedModel for any class with a
`propose(history) -> Action` method to plug in a real LLM.
"""
from __future__ import annotations

import shutil
import subprocess
import sys
import tempfile
from dataclasses import dataclass, field
from pathlib import Path
from typing import Callable, Dict, List, Optional


@dataclass
class Action:
    tool: str                       # read_file | edit_file | run_tests | shell | finish
    args: Dict = field(default_factory=dict)
    why: str = ""


@dataclass
class Step:
    n: int
    phase: str
    text: str


class Policy:
    """Policy check = the orchestration layer's gate, NOT the model's opinion."""
    ALLOWED_TOOLS = {"read_file", "edit_file", "run_tests", "finish"}
    EDITABLE = {"tracker.py"}      # bounded scope: only this file may change

    def check(self, a: Action) -> Optional[str]:
        if a.tool not in self.ALLOWED_TOOLS:
            return f"tool '{a.tool}' is not authorised"
        if a.tool == "edit_file" and a.args.get("path") not in self.EDITABLE:
            return f"edit outside allowed scope: {a.args.get('path')}"
        return None


class Tools:
    def __init__(self, root: Path):
        self.root = root

    def read_file(self, path: str) -> str:
        return (self.root / path).read_text()

    def edit_file(self, path: str, old: str, new: str) -> str:
        p = self.root / path
        text = p.read_text()
        if old not in text:
            return "ERROR: text to replace not found"
        p.write_text(text.replace(old, new))
        return "edited"

    def run_tests(self) -> str:
        cp = subprocess.run([sys.executable, "-m", "unittest", "-q"], cwd=self.root,
                            capture_output=True, text=True, timeout=60)
        out = (cp.stdout + cp.stderr).strip()
        return f"rc={cp.returncode}\n{out}"


class ScriptedModel:
    """Deterministic 'LLM': proposes the next action from the observation history."""

    def __init__(self):
        self.stage = 0

    def propose(self, history: List[Step]) -> Action:
        last = history[-1].text if history else ""
        if self.stage == 0:
            self.stage = 1
            return Action("read_file", {"path": "tracker.py"}, "inspect the filter")
```

Main implementation: agent_loop.py (part 1/2)

```
        if self.stage == 1:
            self.stage = 2
            return Action("shell", {"cmd": "rm -rf ."}, "(bad idea) clean workspace first")
        if self.stage == 2:
            self.stage = 3
            return Action("edit_file", {"path": "tracker.py", "old": 'if status == "all":',
                                        "new": 'status = status.lower()\n    if status == "all":'},
                          "first attempt: lower-case the requested status")
        if self.stage == 3:
            self.stage = 4
            return Action("run_tests", {}, "check focused tests")
        if self.stage == 4:
            self.stage = 5
            return Action("edit_file", {"path": "tracker.py", "old": 't["status"] == status',
                                        "new": 't["status"].lower() == status'},
                          "stored status is mixed case too: normalise both sides")
        if self.stage == 5:
            self.stage = 6
            return Action("run_tests", {}, "re-run after adjustment")
        return Action("finish", {}, "criteria met")


def run_loop(model, workspace: Path, max_steps: int = 12, max_retries_same: int = 1) -> List[Step]:
    tmp = Path(tempfile.mkdtemp())
    root = tmp / "ws"
    shutil.copytree(workspace, root, ignore=shutil.ignore_patterns("__pycache__"))
    tools, policy, log = Tools(root), Policy(), []
    history: List[Step] = []

    def note(phase, text):
        s = Step(len(log) + 1, phase, text)
        log.append(s)
        history.append(s)

    note("TASK", "Done filter shows wrong tasks. Fix with bounded change; verify.")
    last_action, repeats = None, 0
    for _ in range(max_steps):
        action = model.propose(history)
        note("PROPOSE", f"{action.tool} {action.args if action.tool != 'edit_file' else action.args.get('path')} - {ac
        key = (action.tool, tuple(sorted(action.args.items())))
        repeats = repeats + 1 if key == last_action else 0
        last_action = key
        if repeats > max_retries_same:
            note("ESCALATE", "same action repeated with no change -> stop, ask a human")
            break
        blocked = policy.check(action)
        if blocked:
            note("POLICY", f"BLOCKED: {blocked}")
            note("OBSERVE", "action was NOT executed; model must propose something else")
            continue
        note("POLICY", "allowed")
        if action.tool == "finish":
            tests = [x.text for x in log if x.phase == "OBSERVE" and x.text.startswith("rc=")]
            ok = bool(tests) and tests[-1].startswith("rc=0")
            note("VERIFY", "latest test run is green (rc=0)" if ok else "NOT VERIFIED: no passing test run observed")
            note("HANDOFF", "report exact revision + checks + limitations")
            break
        if action.tool == "read_file":
            out = tools.read_file(**action.args)
        elif action.tool == "edit_file":
            out = tools.edit_file(**action.args)
        else:
            out = tools.run_tests()
        note("EXECUTE", f"{action.tool} executed")
        note("OBSERVE", out.replace("\n", " | ")[:300])
    shutil.rmtree(tmp, ignore_errors=True)
    return log
```

Main implementation: agent_loop.py (part 2/2)

demo.py

```
from pathlib import Path
from agent_loop import ScriptedModel, run_loop

print("Agent loop on the Task Tracker 'Done filter' bug (model = scripted stand-in)\n")
for s in run_loop(ScriptedModel(), Path(__file__).parent / "workspace"):
    print(f"{s.n:02d} {s.phase:<8} {s.text}")
```

Demo script

6. Demo and test screenshots

terminal - run the tests

```
$ python -m unittest discover -v
test_dangerous_shell_is_blocked_not_executed (test_agent_loop.LoopTests.test_dangerous_shell_is_blocked_not_executed) ...
ok
test_ends_with_verify_and_handoff (test_agent_loop.LoopTests.test_ends_with_verify_and_handoff) ... ok
test_first_attempt_fails_then_adjusts_then_passes
(test_agent_loop.LoopTests.test_first_attempt_fails_then_adjusts_then_passes) ... ok
test_original_workspace_untouched (test_agent_loop.LoopTests.test_original_workspace_untouched) ... ok
test_policy_scope (test_agent_loop.LoopTests.test_policy_scope) ... ok
test_repeating_same_failing_action_escalates (test_agent_loop.LoopTests.test_repeating_same_failing_action_escalates) ...
ok

----------------------------------------------------------------------
Ran 6 tests in 0.952s

OK
```

Test run (real output)

terminal - python demo.py

```
$ python demo.py
Agent loop on the Task Tracker 'Done filter' bug (model = scripted stand-in)

01 TASK     Done filter shows wrong tasks. Fix with bounded change; verify.
02 PROPOSE  read_file {'path': 'tracker.py'} - inspect the filter
03 POLICY   allowed
04 EXECUTE  read_file executed
05 OBSERVE  """Task Tracker (buggy baseline used by the agent loop demo)."""" |  |  | def filter_tasks(tasks,
status="all"): |     if status == "all": |         return list(tasks) |     return [t for t in tasks if t["status"] ==
status] |
06 PROPOSE  shell {'cmd': 'rm -rf .'} - (bad idea) clean workspace first
07 POLICY   BLOCKED: tool 'shell' is not authorised
08 OBSERVE  action was NOT executed; model must propose something else
09 PROPOSE  edit_file tracker.py - first attempt: lower-case the requested status
10 POLICY   allowed
11 EXECUTE  edit_file executed
12 OBSERVE  edited
13 PROPOSE  run_tests {} - check focused tests
14 POLICY   allowed
15 EXECUTE  run_tests executed
16 OBSERVE  rc=1 | ====================================================================== | FAIL:
test_open_includes_mixed_case_and_excludes_done
(test_tracker.FilterTests.test_open_includes_mixed_case_and_excludes_done) | ----------------------------------------
---------------------- | Traceback (most
17 PROPOSE  edit_file tracker.py - stored status is mixed case too: normalise both sides
18 POLICY   allowed
19 EXECUTE  edit_file executed
20 OBSERVE  edited
21 PROPOSE  run_tests {} - re-run after adjustment
22 POLICY   allowed
23 EXECUTE  run_tests executed
24 OBSERVE  rc=0 | ---------------------------------------------------------------------- | Ran 3 tests in 0.000s |  | OK
25 PROPOSE  finish {} - criteria met
26 POLICY   allowed
27 VERIFY   latest test run is green (rc=0)
28 HANDOFF  report exact revision + checks + limitations
```

Demo output (python demo.py)

7. Evidence-backed report

Change	The Agent Working Loop
Revision tested	content hash `6af4c281d357` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover` and `python demo.py`
Results	**PASS** - Ran 6 tests in 0.811s
Checks not run	A real LLM (ScriptedModel is deterministic so the demo is reproducible)
Known limitations	Policy is a simple allow-list + single editable file; real systems need richer permission models.
Next required action	Human review, then CI on the pull request

8. Practical takeaway

Think 'propose -> execute -> observe -> verify'. Never mix an attempt with a result.